UNIVERSIDAD DON BOSCO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA EN COMPUTACIÓN
DESARROLLO DE SOFTWARE PARA MÓVILES DSM

Tercer Desafío Practico

Aplicación de Lista de Recursos de Aprendizaje

Diseñar y construir una aplicación Android utilizando Kotlin que funcione como un **Sistema de Gestión de Recursos de aprendizaje (libros, videos, artículos de Ingeniería de Sistemas).** La aplicación debe utilizar la arquitectura **MVC (Model-View-Control)** y consumir una API dinámica para gestionar las listas de recursos con soporte completo para operaciones CRUD, autenticación con roles, y persistencia de datos de usuario.

Indicaciones Generales:

1. El desafío es individual.

2. Si hay soluciones similares o años anteriores y código 100% de internet, automáticamente la nota signada será **"0".**

3. Está permitido apoyarse en herramientas de Inteligencia Artificial (IA) para el desarrollo del desafío, pero el objetivo sigue siendo aprender, la IA debe ser un apoyo para el aprendizaje y no un reemplazo de este.

4. El proyecto debe ser desarrollado en **Android Studio con Kotlin** y entregar un Video realizando la defensa (máx. 15 minutos)

5. Todos los textos deben estar **centralizados** en el archivo strings.xml.

```
<resources>
    <string name="app_name">AprendeApp</string>
    <string name="welcome_message">¡Bienvenido a AprendeApp!</string>
    <!-- Agregar los demás textos aquí -->
</resources>
```

6. El repositorio debe contener verbos en todos los commit, y se pide 10 como mínimo. Deben de colocarlo en español o inglés, pero no spanglish.

   add → agregar
   fix → corregir
   update → actualizar
   remove → eliminar
   improve → mejorar
   implement → implementar
   optimize → optimizar
   refactor → refactorizar

7. Los datos deben ser dinámicos API, en formato Json, revisar Guía de Laboratorio Pueden utilizar: **https://mockapi.io/** o **https://railway.com/**

   Posibles campos del JSON, está abierto a su análisis:

```
{
 "recursos": [
  {
   "id": 1,
   "titulo": "Título del Recurso",
   "descripcion": "Descripción detallada del recurso educativo.",
   "tipo": "Tipo de recurso (libro, video, tutorial, etc.)",
   "enlace": "URL o enlace para acceder al recurso completo",
   "imagen": "URL de la imagen ilustrativa del recurso"
  },
  // Más recursos...
 ]
}
```

Funcionalidades

1. Registro y Login

- Registro de usuario con validación segura de contraseñas.
- Login para acceder y Logout para cerrar aplicación.

Validación de Contraseña:

- Mínimo 12 caracteres.
- Al menos una letra mayúscula.
- Al menos una letra minúscula.
- Al menos un número.
- Al menos un carácter especial (! @#$%^&*).

Sistema de Autenticación con Roles:

- Dos roles (**Estudiante** y **Docente**).
- **Estudiante:** Solo puede ver, buscar, marcar como favorito y calificar recursos.
- **Docente:** Debe tener acceso a una pantalla especial para **crear, editar y eliminar recursos** de la lista principal, lo que obligaría a usar las operaciones POST, PUT y DELETE de la API.

2. Gestión de Recursos (Docente)

CRUD completo consumiendo API externa (MockAPI.io o Railway):

1. **Visualización de recursos**
   - Lista en tarjetas con título, descripción corta, tipo, enlace e imagen.
   - Opción de ordenar y filtrar por tipo o título.
2. **Búsqueda**
   - Búsqueda avanzada por ID, título o tipo.
3. **Agregar recurso**
   - Formulario con validación de campos.
4. **Modificar recurso**
   - Formulario prellenado para edición.
5. **Eliminar recurso**
   - Confirmación antes de eliminar.

3. Interacción de Datos (Persistencia de Usuario):

- **Lista de Favoritos:** El usuario debe poder guardar un recurso en una lista de Favoritos que persista localmente o en el backend, mostrando solo los recursos que ha seleccionado.

- **Sistema de Calificación (Rating):** Permitir a los estudiantes calificar los recursos (ej. de 1 a 5 estrellas). Esta calificación debe actualizarse en el backend para mostrar el promedio a todos los usuarios.

4. UI / UX

- Interfaz clara, intuitiva y responsiva.
- Uso de librerías UI
- Feedback visual (toasts, alerts, loaders).
- Navegación

5. Entregables Aula Virtual.

Se entrega en el **Aula Virtual** con el enlace del repositorio **GitHub** que contenga:

- Código fuente
- README.md con el detalle de la aplicación
- Video demostrativo (máx. 15 minutos)

Rúbrica de evaluación:

Criterio	Ponderación
Puntualidad – Entrega a tiempo	10%
Video demostrativo funcional	10%
Interfaz de usuario (UI/UX)	10%
Calidad de código	10%
Funcionalidad	60%
— Registro y Login	10%
— Validación de contraseñas	10%
— Visualización de recursos	5%
— Búsqueda avanzada	5%
— CRUD completo	25%
— Feedback visual (alerts, loaders, animaciones)	5%
Total	100%

Fecha Máxima de Entrega: Sábado 31 Octubre, hasta 10:00pm